

Game Engine Architecture

Jason Gregory

Game Engine Architecture



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Game Engine Architecture

Third Edition

Jason Gregory



CRC Press

Taylor & Francis Group

Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business

AN A K PETERS BOOK

Cover image: 3D model of SpaceX Merlin rocket engine created by Brian Hauger (www.bionic3d.com).

CRC Press
Taylor & Francis Group
6000 Broken Sound Parkway NW, Suite 300
Boca Raton, FL 33487-2742

© 2019 by Taylor & Francis Group, LLC
CRC Press is an imprint of Taylor & Francis Group, an Informa business

No claim to original U.S. Government works

Printed on acid-free paper
Version Date: 20180529

International Standard Book Number-13: 978-1-1380-3545-4 (Hardback)

This book contains information obtained from authentic and highly regarded sources. Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, please access www.copyright.com (<http://www.copyright.com/>) or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. CCC is a not-for-profit organization that provides licenses and registration for a variety of users. For organizations that have been granted a photocopy license by the CCC, a separate system of payment has been arranged.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

Library of Congress Cataloging-in-Publication Data

Names: Gregory, Jason, 1970- author.
Title: Game engine architecture / Jason Gregory.
Description: Third edition. | Boca Raton : Taylor & Francis, CRC Press, 2018.
| Includes bibliographical references and index.
Identifiers: LCCN 2018004893 | ISBN 9781138035454 (hardback : alk. paper)
Subjects: LCSH: Computer games--Programming--Computer programs. | Software architecture. | Computer games--Design.
Classification: LCC QA76.76.C672 G77 2018 | DDC 794.8/1525--dc23
LC record available at <https://lccn.loc.gov/2018004893>

Visit the Taylor & Francis Web site at
<http://www.taylorandfrancis.com>

and the CRC Press Web site at
<http://www.crcpress.com>

Dedicated to
Trina, Evan and Quinn Gregory,

in memory of our heroes,
Joyce Osterhus, Kenneth Gregory and Erica Gregory.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Contents

Preface	xiii
I Foundations	1
1 Introduction	3
1.1 Structure of a Typical Game Team	5
1.2 What Is a Game?	8
1.3 What Is a Game Engine?	11
1.4 Engine Differences across Genres	13
1.5 Game Engine Survey	31
1.6 Runtime Engine Architecture	38
1.7 Tools and the Asset Pipeline	59
2 Tools of the Trade	69
2.1 Version Control	69
2.2 Compilers, Linkers and IDEs	78
2.3 Profiling Tools	99

2.4	Memory Leak and Corruption Detection	101
2.5	Other Tools	102
3	Fundamentals of Software Engineering for Games	105
3.1	C++ Review and Best Practices	105
3.2	Catching and Handling Errors	119
3.3	Data, Code and Memory Layout	131
3.4	Computer Hardware Fundamentals	164
3.5	Memory Architectures	181
4	Parallelism and Concurrent Programming	203
4.1	Defining Concurrency and Parallelism	204
4.2	Implicit Parallelism	211
4.3	Explicit Parallelism	225
4.4	Operating System Fundamentals	230
4.5	Introduction to Concurrent Programming	256
4.6	Thread Synchronization Primitives	267
4.7	Problems with Lock-Based Concurrency	281
4.8	Some Rules of Thumb for Concurrency	286
4.9	Lock-Free Concurrency	289
4.10	SIMD/Vector Processing	331
4.11	Introduction to GPGPU Programming	348
5	3D Math for Games	359
5.1	Solving 3D Problems in 2D	359
5.2	Points and Vectors	360
5.3	Matrices	375
5.4	Quaternions	394
5.5	Comparison of Rotational Representations	403
5.6	Other Useful Mathematical Objects	407
5.7	Random Number Generation	412

II	Low-Level Engine Systems	415
6	Engine Support Systems	417
6.1	Subsystem Start-Up and Shut-Down	417
6.2	Memory Management	426
6.3	Containers	441
6.4	Strings	456
6.5	Engine Configuration	470
7	Resources and the File System	481
7.1	File System	482
7.2	The Resource Manager	493
8	The Game Loop and Real-Time Simulation	525
8.1	The Rendering Loop	525
8.2	The Game Loop	526
8.3	Game Loop Architectural Styles	529
8.4	Abstract Timelines	532
8.5	Measuring and Dealing with Time	534
8.6	Multiprocessor Game Loops	544
9	Human Interface Devices	559
9.1	Types of Human Interface Devices	559
9.2	Interfacing with a HID	561
9.3	Types of Inputs	563
9.4	Types of Outputs	569
9.5	Game Engine HID Systems	570
9.6	Human Interface Devices in Practice	587
10	Tools for Debugging and Development	589
10.1	Logging and Tracing	589
10.2	Debug Drawing Facilities	594
10.3	In-Game Menus	601
10.4	In-Game Console	604
10.5	Debug Cameras and Pausing the Game	605
10.6	Cheats	606

10.7	Screenshots and Movie Capture	606
10.8	In-Game Profiling	608
10.9	In-Game Memory Stats and Leak Detection	615
III	Graphics, Motion and Sound	619
II	The Rendering Engine	621
11.1	Foundations of Depth-Buffered Triangle Rasterization	622
11.2	The Rendering Pipeline	667
11.3	Advanced Lighting and Global Illumination	697
11.4	Visual Effects and Overlays	710
11.5	Further Reading	719
12	Animation Systems	721
12.1	Types of Character Animation	721
12.2	Skeletons	727
12.3	Poses	729
12.4	Clips	734
12.5	Skinning and Matrix Palette Generation	750
12.6	Animation Blending	755
12.7	Post-Processing	774
12.8	Compression Techniques	777
12.9	The Animation Pipeline	784
12.10	Action State Machines	786
12.11	Constraints	806
13	Collision and Rigid Body Dynamics	817
13.1	Do You Want Physics in Your Game?	818
13.2	Collision/Physics Middleware	823
13.3	The Collision Detection System	825
13.4	Rigid Body Dynamics	854
13.5	Integrating a Physics Engine into Your Game	892
13.6	Advanced Physics Features	909

14	Audio	911
14.1	The Physics of Sound	912
14.2	The Mathematics of Sound	924
14.3	The Technology of Sound	941
14.4	Rendering Audio in 3D	955
14.5	Audio Engine Architecture	974
14.6	Game-Specific Audio Features	995
IV	Gameplay	1013
15	Introduction to Gameplay Systems	1015
15.1	Anatomy of a Game World	1016
15.2	Implementing Dynamic Elements: Game Objects	1021
15.3	Data-Driven Game Engines	1024
15.4	The Game World Editor	1025
16	Runtime Gameplay Foundation Systems	1039
16.1	Components of the Gameplay Foundation System	1039
16.2	Runtime Object Model Architectures	1043
16.3	World Chunk Data Formats	1062
16.4	Loading and Streaming Game Worlds	1069
16.5	Object References and World Queries	1079
16.6	Updating Game Objects in Real Time	1086
16.7	Applying Concurrency to Game Object Updates	1101
16.8	Events and Message-Passing	1114
16.9	Scripting	1134
16.10	High-Level Game Flow	1157
V	Conclusion	1159
17	You Mean There's More?	1161
17.1	Some Engine Systems We Didn't Cover	1161
17.2	Gameplay Systems	1162
	Bibliography	1167
	Index	1171



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Preface

Welcome to *Game Engine Architecture*. This book aims to present a complete discussion of the major components that make up a typical commercial game engine. Game programming is an immense topic, so we have a lot of ground to cover. Nevertheless, I trust you'll find that the depth of our discussions is sufficient to give you a solid understanding of both the theory and the common practices employed within each of the engineering disciplines we'll cover. That said, this book is really just the beginning of a fascinating and potentially lifelong journey. A wealth of information is available on all aspects of game technology, and this text serves both as a foundation-laying device and as a jumping-off point for further learning.

Our focus in this book will be on game engine technologies and architecture. This means we'll cover the theory underlying the various subsystems that comprise a commercial game engine, the data structures, algorithms and software interfaces that are typically used to implement them, and how these subsystems function together within a game engine as a whole. The line between the game engine and the game is rather blurry. We'll focus primarily on the engine itself, including a host of low-level foundation systems, the rendering engine, the collision system, the physics simulation, character animation, audio, and an in-depth discussion of what I call the gameplay foundation layer. This layer includes the game's object model, world editor, event system and scripting system. We'll also touch on some aspects of gameplay program-

ming, including player mechanics, cameras and AI. However, by necessity, the scope of these discussions will be limited mainly to the ways in which gameplay systems interface with the engine.

This book is intended to be used as a course text for a two- or three-course college-level series in intermediate game programming. It can also be used by amateur software engineers, hobbyists, self-taught game programmers and existing members of the game industry alike. Junior engineers can use this text to solidify their understanding of game mathematics, engine architecture and game technology. And some senior engineers who have devoted their careers to one particular specialty may benefit from the bigger picture presented in these pages as well.

To get the most out of this book, you should have a working knowledge of basic object-oriented programming concepts and at least some experience programming in C++. The game industry routinely makes use of a wide range of programming languages, but industrial-strength 3D game engines are still written primarily in C++. As such, any serious game programmer needs to be able to code in C++. We'll review the basic tenets of object-oriented programming in Chapter 3, and you will no doubt pick up a few new C++ tricks as you read this book, but a solid foundation in the C++ language is best obtained from [46], [36] and [37]. If your C++ is a bit rusty, I recommend you refer to these or similar books to refresh your knowledge as you read this text. If you have no prior C++ experience, you may want to consider reading at least the first few chapters of [46] and/or working through a few C++ tutorials online, before diving into this book.

The best way to learn computer programming of any kind is to actually write some code. As you read through this book, I strongly encourage you to select a few topic areas that are of particular interest to you and come up with some projects for yourself in those areas. For example, if you find character animation interesting, you could start by installing OGRE and exploring its skinned animation demo. Then you could try to implement some of the animation blending techniques described in this book, using OGRE. Next you might decide to implement a simple joystick-controlled animated character that can run around on a flat plane. Once you have something relatively simple working, expand upon it! Then move on to another area of game technology. Rinse and repeat. It doesn't particularly matter what the projects are, as long as you're practicing the art of game programming, not just reading about it.

Game technology is a living, breathing thing that can never be entirely captured within the pages of a book. As such, additional resources, errata, updates, sample code and project ideas will be posted from time to time on

this book's website at <http://www.gameenginebook.com>. You can also follow me on Twitter @jqgregory.

New to the Third Edition

The computing hardware that lies at the heart of today's game consoles, mobile devices and personal computers makes heavy use of *parallelism*. Deep within the CPUs and GPUs in these devices, multiple functional units operate simultaneously, employing a "divide and conquer" approach to high-speed computation. While parallel computing hardware can make traditional single-threaded programs run faster, programmers need to write *concurrent software* to truly take advantage of the hardware parallelism that has become ubiquitous in modern computing platforms.

In prior editions of *Game Engine Architecture*, the topics of parallelism and concurrency were touched on in the context of game engine design. However, they weren't given the in-depth treatment they deserved. In this, the third edition of the book, this problem has been remedied via the addition of a brand new chapter on concurrency and parallelism. Chapters 8 and 16 have also been augmented to include detailed discussions of how concurrent programming techniques are typically applied to game engine subsystem and game object model updates, and how a general-purpose job system can be used to unlock the power of concurrency within a game engine.

I've already mentioned that every good game programmer must have a strong working knowledge of C++ (in addition to the wide variety of other useful languages used regularly in the game industry). In my view, a programmer's knowledge of high-level languages should rest upon a solid understanding of the software and hardware systems that underlie them. As such, in this edition I've expanded Chapter 3 to include a treatment of the fundamentals of computer hardware, assembly language, and the operating system kernel.

This third edition of *Game Engine Architecture* also improves upon the treatment of various topics covered in prior editions. A discussion of local and global compiler optimizations has been added. Fuller coverage of the various C++ language standards is included. The section on memory caching and cache coherency has been expanded. The animation chapter has been streamlined. And, as with the second edition, various errata have been repaired that were brought to my attention by you, my devoted readers. Thank you! I hope you'll find that the mistakes you found have all been fixed. (Although no doubt they have been replaced by a slew of *new* mistakes, about which you can feel free to inform me, so that I may correct them in the fourth edition of the book!)

Of course, as I've said before, the field of game engine programming is almost unimaginably broad and deep. There's no way to cover every topic in one book. As such, the primary purpose of this book remains to serve as an awareness-building tool and a jumping-off point for further learning. I hope you find this edition helpful on your journey through the fascinating and multifaceted landscape of game engine architecture.

Acknowledgments

No book is created in a vacuum, and this one is certainly no exception. This book—and its third edition, which you now hold in your hands—would not have been possible without the help of my family, friends and colleagues in the game industry, and I'd like to extend warm thanks to everyone who helped me to bring this project to fruition.

Of course, the ones most impacted by a project like this are invariably the author's family. So I'd like to start by offering, for a *third time*, a special thank-you to my wife Trina. She was a pillar of strength during the writing of the original book, and she was as supportive and invaluable helpful as ever during my work on the second and third editions. While I was busy tapping away on my keyboard, Trina was always there to take care of our two boys, Evan (now age 15) and Quinn (age 12), day after day and night after night, often forgoing her own plans, doing my chores as well as her own (more often than I'd like to admit), and always giving me kind words of encouragement when I needed them the most.

I would also like to extend special thanks to my editors for the first edition, Matt Whiting and Jeff Lander. Their insightful, targeted and timely feedback was always right on the money, and their vast experience in the game industry helped to give me confidence that the information presented in these pages is as accurate and up-to-date as humanly possible. Matt and Jeff were both a pleasure to work with, and I am honored to have had the opportunity to collaborate with such consummate professionals on this project. I'd like to extend a special thank-you to Jeff for putting me in touch with Alice Peters and helping me to get this project off the ground in the first place. Matt, thank you also for stepping up to the plate once again and providing me with valuable feedback on the new concurrency chapter in the third edition.

A number of my colleagues at Naughty Dog also contributed to this book, either by providing feedback or by helping me with the structure and topic content of one of the chapters. I'd like to thank Marshall Robin and Carlos Gonzalez-Ochoa for their guidance and tutelage as I wrote the rendering chapter, and Pål-Kristian Engstad for his excellent and insightful feedback

on the content of that chapter. My thanks go to Christian Gyrling for his feedback on various sections of the book, including the chapter on animation, and the new chapter on parallelism and concurrency. I want to extend a special thank-you to Jonathan Lanier, Naughty Dog's resident senior audio programmer extraordinaire, for providing me with a great deal of the raw information you'll find in the audio chapter, for always being available to chat when I had questions, and for providing laser-focused and invaluable feedback after reading the initial draft. I'd also like to thank one of the newest members of our Naughty Dog programming team, Kareem Omar, for his valuable insights and feedback on the new concurrency chapter. My thanks also go to the entire Naughty Dog engineering team for creating all of the incredible game engine systems that I highlight in this book.

Additional thanks go to Keith Schaeffer of Electronic Arts for providing me with much of the raw content regarding the impact of physics on a game, found in Section 13.1. I'd also like to extend a warm thank-you to Christophe Balestra (who was co-president of Naughty Dog during my first ten years there), Paul Keet (who was a lead engineer on the *Medal of Honor* franchise during my time at Electronic Arts), and Steve Ranck (the lead engineer on the *Hydro Thunder* project at Midway San Diego), for their mentorship and guidance over the years. While they did not contribute to the book directly, they did help to make me the engineer that I am today, and their influences are echoed on virtually every page in one way or another.

This book arose out of the notes I developed for a course entitled ITP-485: Programming Game Engines, which I taught under the auspices of the Information Technology Program at the University of Southern California for approximately four years. I would like to thank Dr. Anthony Borquez, the director of the ITP department at the time, for hiring me to develop the ITP-485 course curriculum in the first place.

My extended family and friends also deserve thanks, in part for their unwavering encouragement, and in part for entertaining my wife and our two boys on so many occasions while I was working. I'd like to thank my sister- and brother-in-law, Tracy Lee and Doug Provins, my cousin-in-law Matt Glenn, and all of our incredible friends, including Kim and Drew Clark, Sherilyn and Jim Kritzer, Anne and Michael Scherer, Kim and Mike Warner, and Kendra and Andy Walther. When I was a teenager, my father Kenneth Gregory wrote *Extraordinary Stock Profits*—a book on investing in the stock market—and in doing so, he inspired me to write this book. For this and so much more, I am eternally grateful to him. I'd also like to thank my mother Erica Gregory, in part for her insistence that I embark on this project, and in part for spending countless hours with me when I was a child, beating the art

of writing into my cranium. I owe my writing skills, my work ethic, and my rather twisted sense of humor entirely to her!

I'd like to thank Alice Peters and Kevin Jackson-Mead, as well as the entire A K Peters staff, for their Herculean efforts in publishing the first edition of this book. Since that time, A K Peters has been acquired by the CRC Press, the principal science and technology book division of the Taylor & Francis Group. I'd like to wish Alice and Klaus Peters all the best in their future endeavors. I'd also like to thank Rick Adams, Jennifer Ahringer, Jessica Vega and Cynthia Klivecka of Taylor & Francis for their patient support and help throughout the process of creating the second and third editions of *Game Engine Architecture*, Jonathan Pennell for his work on the cover for the second edition, Scott Shamblin for his work on the third edition's cover art, and Brian Haeger (<http://www.bionic3d.com>) for graciously permitting me to use his beautiful 3D model of the Space X Merlin rocket engine on the cover of the third edition.

I am thrilled to be able to say that both the first and second editions of *Game Engine Architecture* have been or are being translated into Japanese, Chinese and Korean! I would like to extend my sincere thanks to Kazuhisa Minato and his team at Namco Bandai Games for taking on the incredibly daunting task of the Japanese translation, and for doing such a great job with both editions. I'd also like to thank the folks at Softbank Creative, Inc. for publishing the Japanese version of the book. I would also like to extend my warmest thanks to Milo Yip for his hard work and dedication to the Chinese translation project. My sincere appreciation goes to the Publishing House of the Electronics Industry for publishing the Chinese translation of the book, and to both the Acorn Publishing Company and Hongreung Science Publishing Co. for their publication of the Korean translations of the first and second editions, respectively.

Many of my readers took the time to send me feedback and alert me to errors in the first and second editions, and for that I'd like to extend my sincere thanks to all of you who contributed. I'd like to give a special thank-you to Milo Yip, Joe Conley and Zachary Turner for going above and beyond the call of duty in this regard. All three of you provided me with many-page documents, chock full of errata and incredibly valuable and insightful suggestions. I've tried my best to incorporate all of this feedback into the third edition—please keep it coming!

Jason Gregory
April 2018

Part I

Foundations



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Introduction

When I got my first game console in 1979—a way-cool Intellivision system by Mattel—the term “game engine” did not exist. Back then, video and arcade games were considered by most adults to be nothing more than toys, and the software that made them tick was highly specialized to both the game in question and the hardware on which it ran. Today, games are a multi-billion-dollar mainstream industry rivaling Hollywood in size and popularity. And the software that drives these now-ubiquitous three-dimensional worlds—*game engines* like Epic Games’ Unreal Engine 4, Valve’s Source engine and, Crytek’s CRYENGINE® 3, Electronic Arts DICE’s Frostbite™ engine, and the Unity game engine—have become fully featured reusable software development kits that can be licensed and used to build almost any game imaginable.

While game engines vary widely in the details of their architecture and implementation, recognizable coarse-grained patterns have emerged across both publicly licensed game engines and their proprietary in-house counterparts. Virtually all game engines contain a familiar set of core components, including the rendering engine, the collision and physics engine, the animation system, the audio system, the game world object model, the artificial intelligence system and so on. Within each of these components, a relatively small number of semi-standard design alternatives are also beginning to emerge.